Project Design Documentation

1. Project Title & Version Control

Python-Based Automated Web Penetration Testing Framework

Version Control handled by GitHub

2. Project Summary (2-3 sentences)

I am trying to build an Automated Web Penetration Testing Framework based in python. This is a cyber security tool designed to automate common steps used by my chosen role of ethical hacker. This framework will explore online applications for common security risks and return a report of its findings. The project will be tested against intentionally vulnerable web applications such as OWASP (Open Worldwide Application Security Project) Juice Shop and DVWA (**Damn Vulnerable Web Application**).

3. Problem Statement / Use Case

Most common web applications are very large, so checking every aspect of one can take far too long, and parts of the application can be missed if done manually. This project addresses that issue by creating a framework to automate many of the common and repeatable tasks needed for security testing.

The intended user for this type of project would be anyone who needs web applications testing for their security, such as penetration testers, or creators of web applications who need their application testing for security measures.

The framework would be used against applications the user owns or has explicit permission to test, including intentionally vulnerable applications designed for security training.

4. Goals and Objectives

Goal 1 — Automate reconnaissance

Develop a Python-based system that can discover information about a web application.

Goal 2 — Automate vulnerability testing

Implement automated checks for several common web security issues.

Goal 3 — Generate useful security reports

Create an automated reporting system that organizes discovered vulnerabilities.

5. Key Features / Functions

1. Target Input

The user will be able to provide a target URL to scan.

2. Web Crawler

The framework will automatically explore the target application and identify every piece of it to explore and test.

3. Reconnaissance

Collect basic information about the target application.

4. Vulnerability Scanner

The framework performs automated checks for selected vulnerabilities.

5. Security Tool Integration

Where appropriate, the framework can integrate established security tools rather than attempting to recreate every security-testing capability from scratch.

6. Automated Reporting

Generate an HTML report containing every finding of the framework.

7. Logging

Maintain logs showing what the framework did during the scan.

6. Tech Stack and Tools

Technology	Purpose
Python	Primary programming language and framework development
Git	Version control
GitHub	Source-code management and project documentation
Linux	Development and security-testing environment
Docker	Running intentionally vulnerable web applications
Requests	Sending HTTP requests from Python
BeautifulSoup	Parsing HTML and discovering links/forms
Selenium/Playwright	Optional browser automation for JavaScript-heavy applications
Nuclei	Automated vulnerability/template-based security testing
OWASP ZAP	Optional web application security scanning
Nmap	Optional reconnaissance/network discovery
Jinja2	Generating HTML reports
SQLite/JSON	Storing scan results

OWASP Juice Shop	Vulnerable application used for testing
DVWA	Additional vulnerable application used for testing

7. Architecture / Workflow Diagram

Target URL

↓

Reconnaissance

↓

Crawl Application

↓

Identify Endpoints & Parameters

↓

Run Security Tests

↓

Collect Findings

↓

Determine Severity

↓

Generate Report

↓

Review Results

8. Timeline / Weekly Milestones

Week	SDLC Phase	Tasks
1	Planning	Define project scope, requirements, MVP features, testing targets,

		and project architecture
2	Research & Design	Research OWASP vulnerabilities, penetration-testing methodology, Python libraries, and existing security tools
3	Environment Setup	Set up Python project, Git repository, Docker, Juice Shop/DVWA, and basic project structure
4	Development	Build command-line interface and target input functionality
5	Development	Implement basic HTTP requests and reconnaissance functionality
6	Development	Build web crawler and endpoint/parameter discovery
7	Development	Implement initial vulnerability checks
8	Development	Add additional vulnerability checks and improve detection accuracy
9	Integration	Integrate selected tools such as Nuclei or OWASP ZAP
10	Reporting	Build finding-management system and automated HTML report generation
11	Testing	Test against vulnerable applications, identify false positives/negatives, fix bugs, and improve performance

12	Release	Final testing, documentation, screenshots/demo, cleanup, GitHub publication, and final presentation

9. Risks and Risk Mitigation

The Risks	Potential Impact	Mitigation
Project becomes too large	Important features may not be completed	Establish a small MVP and prioritize core scanning/reporting functionality
Vulnerability detection is difficult	Scanner may produce inaccurate results	Start with simple checks and compare results against known vulnerable applications
False positives	Framework reports vulnerabilities that aren't actually present	Include evidence with findings and manually validate results
False negatives	Framework misses vulnerabilities	Clearly define the limitations of automated scanning and test against known vulnerabilities
External tools are difficult to integrate	Development delays	Make external tools optional and ensure the core Python framework works independently
Time constraints	Project may remain unfinished	Follow weekly milestones and avoid adding future features until MVP functionality is complete
Target application changes	Tests may stop working	Use controlled, intentionally vulnerable applications for consistent testing

Accidentally scanning unauthorized systems	Potential legal/security issues	Only test local applications or systems where explicit authorization exists
Technical complexity	Some security concepts may be difficult to implement	Research one vulnerability at a time and build individual modules before combining them

10. Evaluation Criteria

1. The framework can accept a target URL, crawl the application, perform its configured security checks, and complete a scan without crashing.
2. The framework can properly detect predefined vulnerabilities in tested applications.
3. The framework automatically produces a readable HTML security report after a scan.
4. Running the framework against the same test application should produce consistent results under the same conditions.
5. The completed project can be demonstrated from start to finish.

11. Future Considerations

Potential improvements to this project could include:

- Expanding into a larger cybersecurity lab containing multiple intentionally vulnerable applications.
- Detecting more advanced vulnerabilities.
- Making the framework more user friendly